Copyrighted Materials
Copyright © 2023 American Institute of Aeronautics and Astronautics
Retrieved from app.knovel.com

Model-Based Systems Engineering for AI-Based Systems

Jasper Sprockhoff* Bojan Lukic† Vincent Janson‡ Siddhartha Gupta§ Alexander Ahlbrecht¶ Umut Durak‖
German Aerospace Center (DLR), Braunschweig, Germany

Thomas Krueger**
German Aerospace Center (DLR), Sankt Augustin, Germany

In recent years, there has been significant progress in Artificial Intelligence (AI), leading to an increasing interest for integration of AI-based functions into newly developed systems. AI promises several benefits, amongst others, beyond the state-of-the-art functions and performance. However, the use of AI-techniques also introduces new challenges regarding safety and security of systems and their certification. These challenges mostly originate from the "black box nature" of complex AI algorithms. To tackle the challenges, safety of the AI-based systems has to be addressed throughout the entire development and life cycle of the system. The adaption of existing methods to the development of AI-based systems is necessary. An established method for the development of complex systems is Model-Based Systems Engineering (MBSE), which offers several advantages for the systems engineering process. In this paper three application examples of how MBSE can support the engineering process of AI-based systems are presented using an application use case: An AI-based threat localization system. First, a systematic development framework is used to design and model the AI-based system. Second, it is demonstrated how safety analysis can be integrated into a model of the system to identify potentially hazardous scenarios, which could arise, for example, due to erroneous predictions by an AI. For the analysis, an approach called Model-Based STPA is utilized which is based on the System-Theoretic Process Analysis. Third, it is demonstrated how MBSE can help in performing scenario-based safety assessment. From the operational domain model, executable configurations are generated to run scenario-based test cases.

I. Introduction

For the last few years, Artificial Intelligence (AI) has been significantly gaining importance in the development of Cyber-Physical Systems. For many activities, it is predicted that AI-based systems will be able to perform significantly better than a human could do in the near future [1]. It is apparent that the aviation industry could also greatly benefit from utilizing AI techniques. For instance, in [2] it was shown how a deep neural network replaces the large numerical table of an airborne collision avoidance system and reduces the required storage space by a factor of 1000. However, while the introduction of AI into the safety-critical aircraft systems offers new opportunities, it also presents significant challenges that must be overcome. According to [3], the main challenges to certifying AI-based systems arise from problems found during four different development steps: the specifications, the data constitution, the learning process, and the implementation stage. As further stated in [3], existing standards in the aviation domain such as ARP4754A [4] and DO-178C [5] cannot simply be updated or complemented to address the AI-specific problems. Therefore, new standards and guidelines regarding the development of AI-based systems are crucial. Currently, researchers are working toward the necessary steps to tackle these challenges and enable the usage of AI-based systems in aviation [6, 7]. Amongst many AI techniques, recent focus is towards Machine Learning (ML). The European Aviation Safety Agency (EASA) is aiming for initial certifications of ML-based system for pilot assistance by 2025, while the deployment of fully autonomous aircraft that would probably utilize multitude of AI techniques is targeted for 2035 [8]. Besides a regulatory framework, systematic approaches to safely integrate AI-based components into complex systems while assuring safety-critical functions need to be created.

*Research Scientist, Institute of Flight Systems.
†Research Scientist, Institute of Flight Systems.
‡Research Scientist, Institute of Flight Systems.
§Research Scientist, Institute of Flight Systems.
¶Research Scientist, Institute of Flight Systems.
‖Group Leader, Institute of Flight Systems, AIAA Associate Fellow.
**Deputy Director, Institute of AI Safety and Security.

The rising complexity of systems today requires the use of systematic procedures within the development. One approach to the system development process that has become increasingly relevant in recent years is Model-Based System Engineering (MBSE) [9]. Unlike traditional document-based systems engineering techniques, MBSE uses a centralized and standardized model to manage system development. Formalizing and centralizing system development activities reduces consistency problems that can occur with distributed, document-based development by providing a common interpretation and a consistent view on elements [10]. It also enables the allocation of requirements to functional blocks within a model and simplifies the tracing of requirements through the different levels of system architecture. In addition, modern MBSE tools allow for data import and export in order to transfer knowledge between different projects. Another major advantage of MBSE is the ability to automate various processes, which is made possible by the formalization of the models. Automating processes is especially useful when the model is changed frequently, as it saves time and reduces the risk of errors caused by human mistakes. The implementation of ML applications is a process that is subject to frequent changes because the development of an ML application is an iterative process consisting of data collection, training, and evaluation until the targeted performance is achieved. Therefore, the utilization of state-of-art MBSE techniques for the development of AI-based systems is desirable.

This paper presents possibilities of how MBSE can help during the development process of AI-based systems, with a focus on the automation aspect. The ideas are demonstrated with the example of a computer vision-based threat localization system, that has been modeled using the Systems Modeling Language (SysML). First, an MBSE framework is used to systematically design and model the system. Then, it is shown how Model-Based Safety Analysis (MBSA) can be conducted early in the development process. For this purpose, an approach called Model-Based STPA is used to identify potentially dangerous situations already in the specification phase of system design. Finally, it is shown how MBSE can help performing scenario-based safety assessment by generating executable scenarios, in the use case for the open-source flight simulator FlightGear [11] from a domain model created in SysML.

In the following Section II relevant background information about the current efforts on the introduction of AI into aviation is provided and related work to this paper is introduced. In the next Section III the concept of our project *MBSE4AI* is explained. Afterwards, in Section IV the proposed approaches to how MBSE can support the development of AI-based systems regarding the topics of *Systematic Design Frameworks* (IV.A), *Model-Based Safety Analysis* (IV.B) and *Scenario-Based Safety Assessment* (IV.C) are demonstrated. Section V concludes the paper and provides a brief outlook on our future research regarding the topic of combining MBSE and AI.

II. Background & Related Work

The purpose of this section is to provide information to situate the paper within current efforts of introducing AI into avionics. First, an overview of the current efforts is provided, followed by an overview of the literature related to our work.

A. Current Efforts on Machine Learning in Aviation

To introduce ML into the aviation domain, EASA has taken a leading role by providing guidelines and specific objectives for the development of AI-based systems for safety-critical applications. Hereby, EASA's *Artificial Intelligence Roadmap* [8] serves as the baseline. The EASA has defined *AI trustworthiness analysis*, *learning assurance*, *explainability* and *safety risk mitigation* as the four essential building blocks to create trust in an AI-based system that must be studied in depth. The ultimate goal stated in the roadmap is to operate autonomous commercial air transport operations by 2035 [8]. As an early step, a collaboration between EASA and Daedalean was initiated with the goal to identify challenges posed by the introduction of AI-based systems into safety-critical avionics applications. Within the scope of the collaboration, a report titled "Concepts of Design Assurance for Neural Networks" (CoDANN) [12] was published in 2020. The main contributions of the report were the definition of the *W-shaped learning assurance life-cycle* as an adaptation of the V-model, the investigation of the notion of "generalization" of neural networks, and the formulation of an approach to consider neural networks in safety assessment. In the following year, another collaboration between EASA and Daedalean was launched resulting in the publication of a second report on CoDANN [13], titled CoDANN2. The second report focuses on maturing the learning assurance and trustworthiness analysis concepts from the first report and discussing the explainability building block. Furthermore, the EASA has proposed a possible classification of AI/ML applications consisting of three different levels [8]. While applications for human assistance are considered Level 1, Level 2 are applications for human/machine collaboration and Level 3 are autonomous machines. A higher level requires higher safety and stricter certification. For Level 1 applications the EASA released the first usable guidance for Level 1 ML applications in 2021 [6] and a guideline for Level 2 ML applications was set to follow up in

the end of 2022. It is expected that the goals stated in the guidelines will be updated as the research progresses. In parallel to the work of EASA, the European Organization for Civil Aviation Equipment (EUROCAE) working group WG-114/Artificial Intelligence was created in 2019. WG-114 collaborates with SAE G-34 to work on a new standard titled AS6983 for development and certification of aeronautical systems including ML. AS6983 includes a newly created Machine Learning Development Lifecycle (MLDL) to guide the development [14]. EUROCAE WG-114 and SAE G-34 also published an aerospace information report in 2021 titled "Artificial Intelligence in Aeronautical Systems: Statement of Concerns" [7]. The key deliverable of the report are "documented standards, providing guidance on assuring safe and secure systems utilizing AI, through an agreed acceptable means of compliance with regulatory requirements" [7]. The report includes a classification of AI techniques, a gap analysis from existing standards, and potential next steps for the implementation of AI applications in the aviation domain. The Dependable and Explainable Learning (DEEL) research program was initiated by manufacturers from multiple domains including aeronautics with the goal to create knowledge on the explainability and robustness aspects of AI algorithms. In a white paper published in 2021, seven main challenges, each consisting of several sub-challenges, are discussed which need to be addressed in order to achieve certification in the future [15]. More information about upcoming regulations and standards for certification of AI-based systems, specifically in the aviation domain, can be found in [16]. An overview of general literature on the subject of machine learning verification and safety is provided in [17].

B. Related Work

1. Systematic Design Framework

There are a variety of MBSE methodologies available to guide the user through the MBSE process [18]. Currently popular techniques include the Object-Oriented Systems Engineering Method (OOSEM), which is using object-oriented concepts in combination with traditional systems engineering practices for system development [19]. In addition, the Systems Modeling Toolbox (SYSMOD) methodology provides methods well-suited for the popular modeling language SysML [20]. Another exemplary MBSE methodology is Arcadia, which is supported by the modeling tool Capella, which was specifically designed for the method [21]. We applied the rather new MagicGrid framework [22] for the creation of our models. MagicGrid is a comparatively light weight framework, that still includes important aspects such as the division into problem and solution domain. It was developed by NoMagic, who also developed the used modeling tool, the Cameo Systems Modeler, a popular tool for MBSE.

2. Model-Based Safety Analysis using STPA

When developing a system using MBSE, SysML is often used. Currently, however, SysML does not provide any constructs to capture safety information about the system [23]. To address this issue the Object Management Group (OMG) created a specification called Risk Analysis and Assessment Modeling Language (RAAML) that extends SysML with safety analysis methods [23]. RAAML supports different safety approaches, one of them being the Systems-Theoretic Process Analysis (STPA) [24, 25]. The STPA differs from previous safety analysis approaches by focussing on the interactions between system components to identify dangerous scenarios before they occur. Previously to the release of RAAML the authors of [26] have already proposed a method to formally verify SysML models with regard to requirements derived by an STPA. In [27] it was shown how SysML, STPA, and formal methods can be combined. This approach, called Model-Based STPA, allows automating parts of the analysis by generating analysis artifacts such as lists of unsafe control actions and system requirements. Furthermore, in [28] it was demonstrated how Model-Based STPA can be used for safety trade-off analyses early in the development process. In subsequent publications, the model-based implementation of the STPA was enhanced with automatic support for change impact [29] and coverage assessments [30].

3. Scenario-Based Safety Assessment

The scenario-based approach outlined in this paper is adapted from our previous work [31], which focuses on scenario-based data generation process inspired by the EASA guidance for level 1 ML applications [6]. It showed that a metamodel could be used to build a domain model deriving the ODD and scenarios from it. The modelling of scenarios using UML and Ecore [32] inspires the SysML approach used in this paper. Scenarios consist of basic constructs used in [33] dividing a simulation scenario for aviation into entities, environment and events. The description of ODD features in [34] leads to the format used in this paper.

During our research, we found that there is a lack of literature dealing with the use of MBSE in the development of AI-based systems or even proposing concrete concepts. Proposals and ideas exist on certain aspects, that could be integrated into a holistic process. With our work, we want to show further ideas that can help to create development techniques for AI-based systems in safety-critical domains.

III. Resulting Concept

At the present time, little research has been done on the topic of using MBSE to develop AI-based systems or introducing these into complex system architectures. According to [10], however, the aerospace industry is among the industries that profit the most from the adoption of MBSE because of the high complexity and long lifespans of aircraft systems. The high potential of MBSE approaches and the current establishment of new initiatives for certification of AI in avionics have prompted the *Institute of Flight Systems* and the *Institute for AI Safety and Security* of the *German Aerospace Center* (DLR) to collaborate on a newly launched project titled "*Model-Based Systems Engineering for Artificial Intelligence*" (MBSE4AI). MBSE4AI started in 2022 with the goal to identify and implement concepts for development and safety assurance of AI-based systems using MBSE. We have specified multiple potential intersections between the two domains that will be explored within the scope of MBSE4AI. Figure 1 provides an overview of our proposed interfaces.

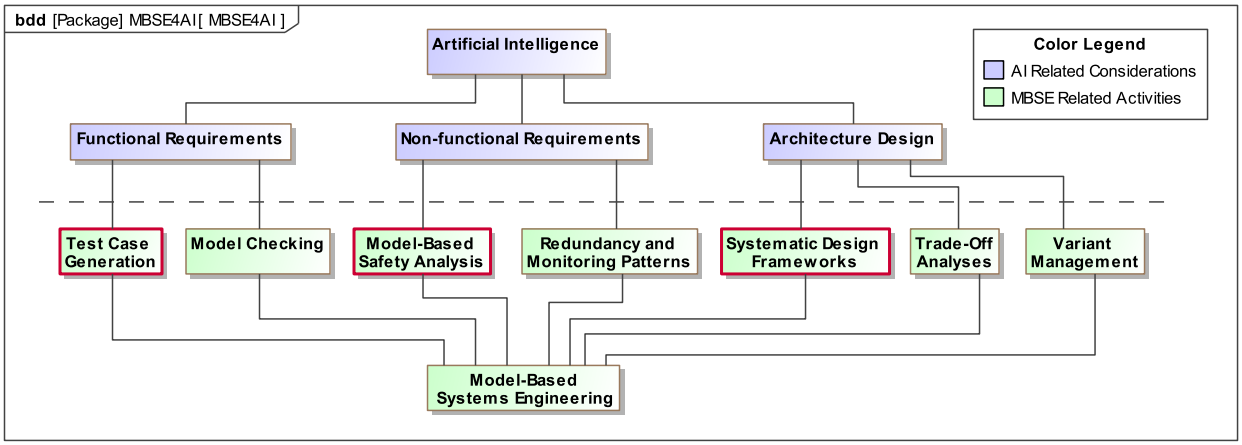

Fig. 1 Relationship between engineering AI/ML-based systems and MBSE activities

The first intersection explores how to perform development of AI-based systems using an MBSE approach. For example, MBSE can help to allocate, trace, and verify functional requirements of an AI-based system. The model can be used to systematically generate test cases to verify the AI algorithm. In addition, the semi-formalized structure of MBSE models enables model checking so that errors can easily be identified. As the second intersection point it is investigated how MBSE can help in the verification of non-functional requirements of AI-based systems. Conducting MBSA can help to identify dangerous scenarios that may arise during the operation of an AI-based system. Furthermore, additional safety can be achieved by applying redundancy and monitoring patterns. The last intersection identified concerns the comparison of architectures of AI-based systems based on models. Through trade-off analysis and variant management techniques, MBSE can help identify the most appropriate architecture considering the AI component. For the creation of the models, systematic design frameworks should be utilized for which an example is given in this paper. This paper also presents early results of MBSE4AI regarding *Model-Based Safety Analysis* and *Test Case Generation*. Thus, one aspect for each main intersection point is highlighted. The identification of concepts and an associated demonstration of them form the contributions of this paper.

IV. Demonstration

In this section three of the connections between MBSE and AI introduced in Section III are addressed. In the first Subsection IV.A a systematic design approach is applied to create models for an exemplary AI-based system for threat localization. In the following Subsection IV.B the Model-Based STPA is used to identify loss scenarios in the

specification phase of the design of our threat localization system. In the third Subsection IV.C our ideas are shown how MBSE can be used to systematically generate executable scenarios for a demonstrator system.

A. Systematic Design

To study the application of MBSE for the development of AI-based systems we are developing a threat localization system based on computer vision as a case study. For the modeling of the system SysML is being used. As of today, SysML is one of the most used languages for MBSE [35]. It is based on the Unified Modeling Language (UML) and includes nine different diagrams to model behavior and structure of a system as well as the requirements. In contrast to UML, which is used for software development, SysML is better suited to model cyber-physical systems. In addition to the modeling language, MBSE also requires a methodology. We have chosen to use the MagicGrid framework by NoMagic. An appropriate methodology and an extensive modeling tool are the foundations for the successful application of MBSE. They enable the creation of graphical descriptions of complex systems and a clear tracing of requirements to the corresponding components. The following subsections describe the process of modeling the threat localization system. First, the problem domain of the system is modeled, which does not cover the physical architecture of the system. Then, a potential solution to the specified problem is modeled. Finally, our implementation of a demonstrator application is described.

1. Problem Domain

The task of our system is to help the pilot to avoid collisions with other vehicles in an urban airspace scenario. Models of the use case and the related activities are displayed in Fig. 2.

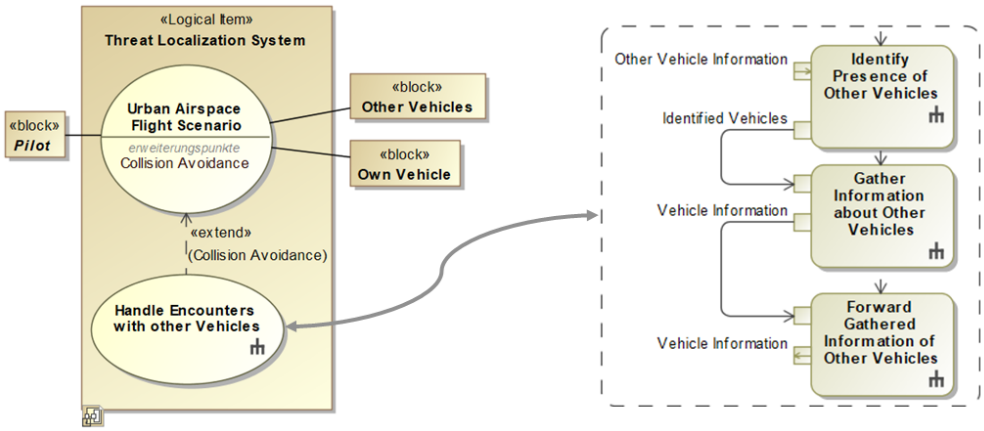

Fig. 2 The use case of our system split into activities

To provide assistance, the system should be able to detect encounters with other vehicles (intruders) and classify them as dangerous or harmless. We have defined three necessary activities: First of all, an intruder must be perceived. Then the system gathers information about the intruder (i.e. position, movement, ...) to assess if a dangerous situation can arise and finally, the system forwards the information gathered to another component that deals with the situation. This component can be, for example, a collision avoidance system, that finds the correct advisory or it could be the pilot himself. However, this other component is not part of our system. As our system shall only provide assistance to the pilot, it can then be considered AI/ML Level 1 according to the EASA classification. Therefore, the objectives set in the Guideline for Level 1 Machine Learning provided by EASA can be referred to when creating the system.

From the activities, a functional architecture of the system is derived. The resulting functional items are then mapped to logical components. In Figure 3, the mapping of functional items to a logical architecture of the threat localization system is displayed. For the identification of an intruder, some kind of a sensor is needed. To gather the information about the identified intruder and conduct a risk analysis, computational components are necessary. Risk assessment is done by localizing and tracking the intruder's position, such that a potential undercutting of a minimum required safety distance can be detected. Also some component is required to forward the threat information depending on the results of our risk assessment. From the logical architecture of the problem domain, solution architectures can be derived. One possible solution would be using cameras as sensors and a convolutional neural network for object detection. However,

modeling the problem domain enables the creation of multiple solution domain models for later comparison through trade-off analyses. The model of the problem domain can be used to conduct safety analyses and identify hazards early in the development process to find additional requirements. Section IV.B shows how the problem domain model of the system is used to apply Model-Based STPA and identify loss scenarios.

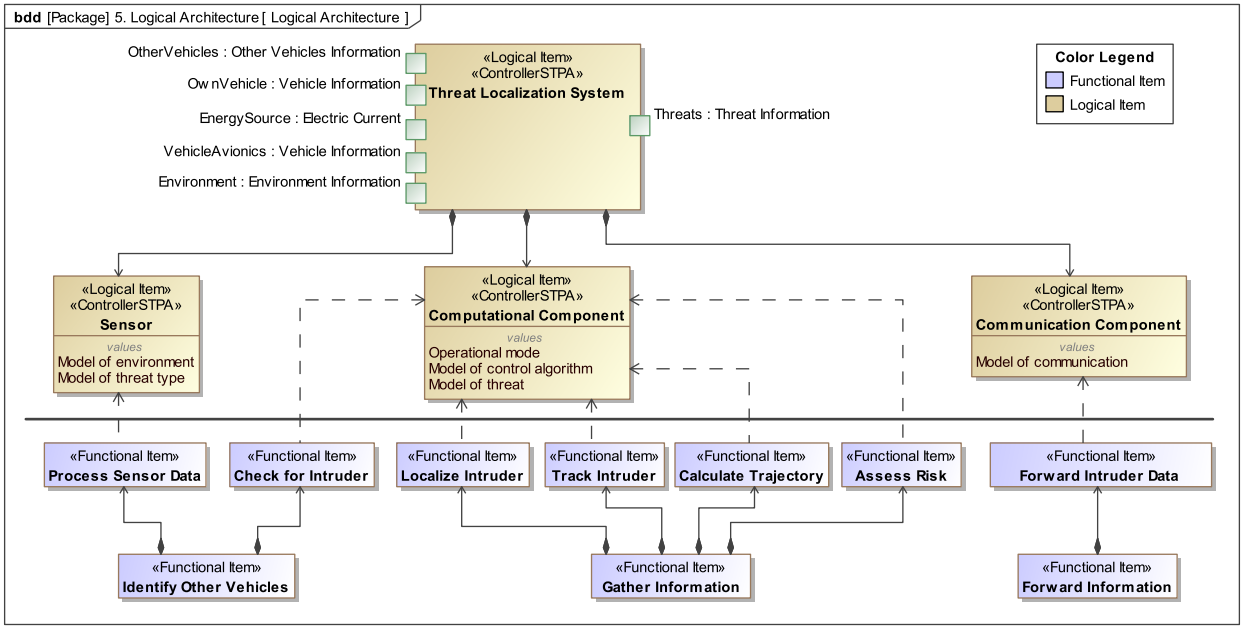

Fig. 3 Initial functional architecture mapped to the logical architecture of the threat localization system

2. Solution Domain

In the solution domain, potential implementations of the system are specified containing more concrete information regarding the individual components. Figure 4 displays the internal architecture of one possible solution. In the presented solution a neural network is responsible for object detection. To run the neural network for inference an embedded system is used. To generate the required sensor inputs multiple cameras are used which provide images as input for the neural network. The data output of the neural network contains information about the positions of intruders. This information is used for risk assessment of each encounter. Depending on the risk assessment, the information about the threat is forwarded out of the system in an appropriate output format. It should be noted that this model is an early design that will be further developed in future work. To apply the system in practice, further work would be necessary to meet certification standards.

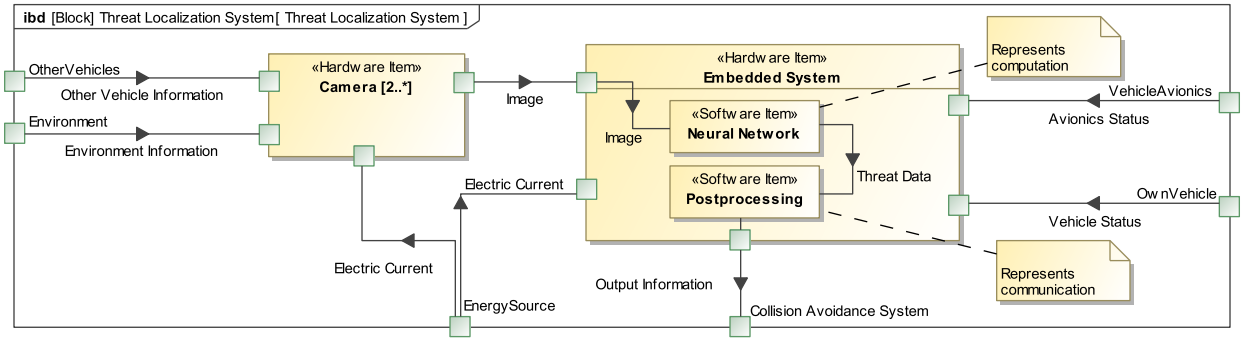

Fig. 4 Initial solution architecture concept of the threat localization system

3. Demonstrator

In addition to the model, we are working on an executable implementation of the threat localization system. This implementation serves as a demonstrator for our ideas of using MBSE concepts for AI development and verification. Our system operates in a virtual environment provided by the open-source flight simulator FlightGear [11]. Virtual cameras are defined on the wings of the aircraft to capture images of the environment in front of the aircraft. The use of multiple cameras placed at a fixed distance from each other allows distance determination. To run simulations, we set up scenarios where different other aircraft enter the flight path of the aircraft. During the execution of the scenarios, the images captured by the cameras are forwarded as inputs to the object detector component, which identifies intruders. The task of identifying intruders is performed by the Yolov7 [36] convolutional object detection neural network. The result of the object detection is so-called bounding boxes that contain information about the position of the intruders in the images. From the positions of the bounding boxes in successive images and the depth information provided by the multiple cameras further information about the trajectory of the intruders can be derived. Taking into account the current trajectory and speed of the own plane, a fall below the minimum allowed distance between the two aircraft can be detected. The distance is used to assess the risk posed by an intruder.

We are running an instance of FlightGear on a Windows Computer. The images generated by FlightGear are sent via Ethernet to an NVIDIA Jetson AGX Xavier which is executing the object detection algorithm. We decided to use the Jetson AGX Xavier to emulate the usage of an embedded system in a real aircraft. After finishing the detection on an image, all identified bounding boxes are returned to the PC running FlightGear. Warnings of dangerous encounters are presented by displaying the bounding boxes of hazardous encounters over the analyzed images. Figure 5 displays an exemplary screenshot from our demonstrator.

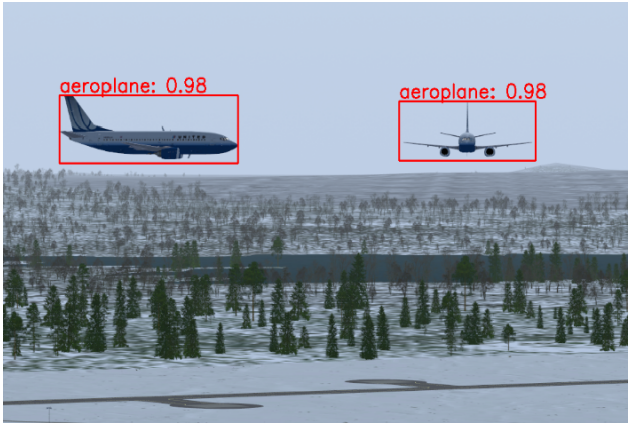

Fig. 5 Image from our object detection-based threat avoidance system in FlightGear

In addition to the study of creating the demonstrator using MBSE, it provides other opportunities to conduct research on AI-based systems. Especially the aspect of execution and analysis of scenarios for the evaluation of self-trained neural networks is a noteworthy possibility here. In Section IV.C it is shown how a SysML model is used to generate configuration files for the demonstrator in order to execute scenario-based safety assessment.

B. Model-Based Safety Analysis

For the development of safety-critical applications, an extensive analysis is essential to assure functional safety. Guidelines, such as ARP4761 (Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment) [37] provide recommended practices on safety assessment processes for the aerospace domain. Integrating an AI component, however, adds new challenges to the safety assurance process that are not addressed in current standards. ML applications are trained and verified on a limited amount of data that usually cannot cover all situations that may occur during real operation. This issue is known as the generalization gap. Because of the generalization gap we can never be completely sure that the AI will always make the correct decision while operating in unknown environments. To assure safety, it is required to identify and investigate scenarios in which the AI makes an inadequate decision in order to assess and mitigate the consequences. By identifying such scenarios, we are able to derive new requirements and adapt the systems structure. The presence of the AI component plays an influential role in the systems design process. Therefore, it is necessary to analyse safety as early in the development process as possible,

since a later redesign is usually associated with loads of work and high costs.
Traditionally safety analyses techniques are the Failure Mode and Effects Analysis (FMEA) and the Fault Tree Analysis (FTA). However, FMEA and FTA are based on component failures, therefore it is difficult to identify safety hazards originating from inadequate user inputs [38]. In the case of an AI-based system these inadequate inputs could be erroneous predictions of a neural network. When neural networks are used to make decisions in unknown environments, the occurrence of such inadequate instructions provided by the AI must be expected. As observed by [39] for neural networks (such as perception networks) currently accuracies of around 99 % are considered very good which is far below the 10^{-9} probability of failure required for aircraft systems. Identification of control actions that result from the AI component and affect system safety is necessary. The STPA is an hazard analysis technique that is capable of identifying inadequate control actions that may cause accidents [24]. The STPA is based on STAMP (System-Theoretic Accident Model and Processes), an accident causality model based on systems theory [25]. To conduct the STPA following steps must be performed [25]:

1) Define Purpose of the Analysis
2) Model the Control Structure
3) Identify Unsafe Control Actions
4) Identify Loss Scenarios

First, potential losses (unwanted consequences) and hazards (potentially dangerous system states) of the system are defined. Next, a hierarchical control structure of the system is modeled displaying the control actions and feedback actions between the individual components. Here, a clear interface between the STPA and MBSE becomes apparent, as both are based on a model of the system. Once the purpose is defined and the control structure is modeled, the preparations for the actual analysis are complete. Now, the control actions are carefully investigated by means of a template of guidewords. It is looked at whether the control action can be unsafe in some contexts. A control action may be unsafe if provided, not provided, or provided at the wrong time or with the wrong duration. Finally, it is investigated how each unsafe control action could occur. For each unsafe control action, individual loss scenarios are created to find appropriate safety measures. To receive more detailed information about the STPA we refer the reader to [24, 25]. For the safety analysis of our threat localization system, an analysis approach titled Model-Based STPA was used. With the goal to guarantee safety by construction Model-Based STPA combines MBSE, STPA, and formal methods [27]. Model-Based STPA allows to automate the generation of safety artifacts through formalization of the STPA. More precisely Model-Based STPA can generate hazard tables that the analyst can then use to conduct the analysis. Model-Based STPA is also capable of generating requirements of the system using STPA-specific word constructs as well as loss scenarios. To perform the Model-Based STPA, corresponding stereotypes must be assigned to the components to be examined. Next, it is demonstrated how the Model-Based STPA can help to identify a loss scenario introduced by a faulty algorithm (e.g. a poorly trained neural network) by showing a simple example using the model of our threat localization system.

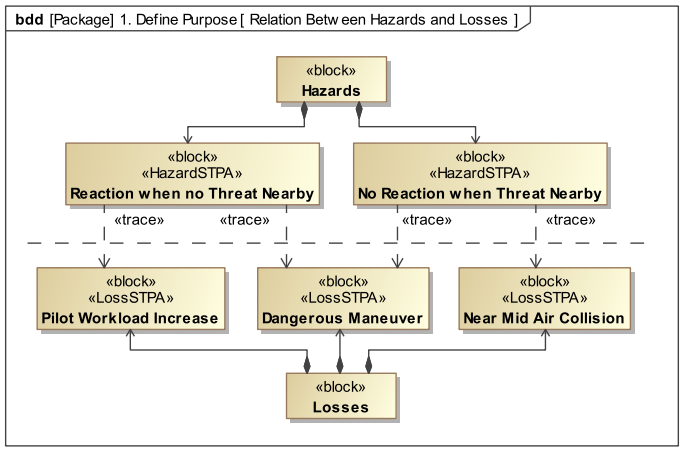

Fig. 6 Identified hazards and relation to losses

In the first step, the scope of the analysis was set by defining the losses and hazards. In our analysis three types of losses were considered: *Near Mid Air Collision*, *Pilot Workload Increase*, and *Dangerous Maneuver*. As possible

hazards two cases were considered: that an existing threat is not detected (*No Reaction when Threat Nearby*) and that a false warning of a threat is given (*Reaction when no Threat Nearby*). Figure 6 shows the losses and hazards with the assigned stereotypes "LossSTPA" and "HazardSTPA".

In the second step, the control structure of the system was modeled as displayed in Fig. 7. The individual components of the system are connected via control actions and feedback. To show the hierarchical structure, control actions are represented as downward arrows and feedback as upwards arrows as it is suggested in [25]. In this exemplary analysis, only the "Threat Data" control action between the computational component and the communication element was considered. With this message, the information about risky encounters is issued to be forwarded to a receiving component outside the system.

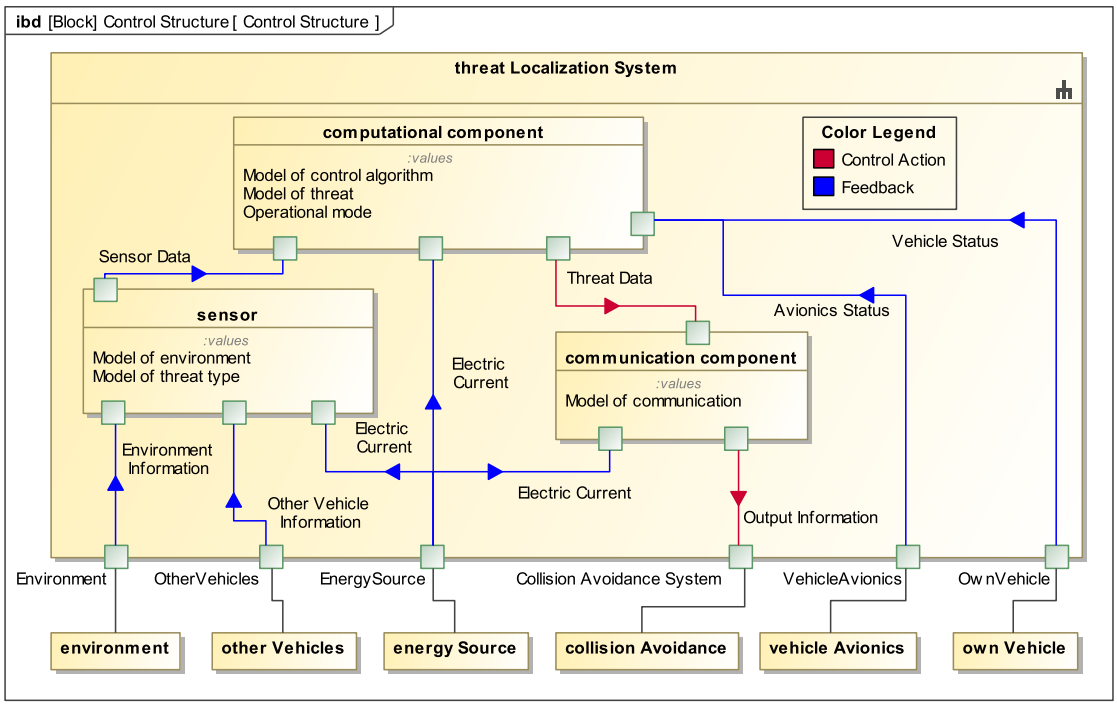

Fig. 7 Control structure of the systems problem domain.

In the analysis, the system was considered in two different contexts: *Threat Present* and *No Threat Present*. With regard to these contexts, an automation function provided by the Model-Based STPA profile was used to generate the table of STPA elements shown in Fig. 8. The elements listed in the table were used to identify the unsafe control actions in the system. In this example, only two types of unsafe control actions were considered: provided and not provided. The STPA elements were analyzed and the potential hazards were assigned depending on the context. An exemplary hazard would be that if a threat is present and the Threat Data is not sent to the communication component, the system will not take any action even though a threat is nearby. Furthermore, the Model-Based STPA was used to generate system requirements. As an example, for row 1 in Fig. 8, the following requirement was generated: "*The [Computational Component] shall always be able to apply [Threat Data] to the [Communication Component] when [Threat Present] is in state [Yes]*". After specifying the hazards and generating the requirements, a table of the unsafe control actions was automatically generated. The resulting table is shown in Fig. 9.

#	Name	Context	Context Value	Source	Receiver	Action	Not provided in this context	Provided in this context
1	STPA Element 0	Threat Present	Yes	Computational Component	Communication Component	Threat Data	No Reaction when Threat Nearby	
2	STPA Element 1	Threat Present	No	Computational Component	Communication Component	Threat Data		Reaction when no Threat Nearby

Fig. 8 Exemplary elements generated for the analysis of the system

#	Name	UCA Description	Hazard Reference	Feedback Variables	Process Variables	Loss Scenarios
1	NoThreatDataWhen ThreatPresent	The [Computational Component] does not apply [Threat Data] to the [Communication Component] when [Threat Present] is in state [Yes].	No Reaction when Threat Nearby	Sensor Data Electric Current Avionics Status Vehicle Status	Model of threat Operational mode Model of control algorithm	Inadequate Sensordata leads to missing identification Faulty algorithm cannot identify threats
2	ThreatDataWhenNo ThreatPresent	The [Computational Component] applies [Threat Data] to the [Communication Component] when [Threat Present] is in state [No].	Reaction when no Threat Nearby	Sensor Data Electric Current Avionics Status Vehicle Status	Model of threat Operational mode Model of control algorithm	Faulty algorithm cannot identify threats Wrong Vehicle Information

Fig. 9 Exemplary unsafe control actions identified during the analysis

After identifying the control actions, the reasons that could cause the respective unsafe control action were analyzed. It was identified that control action 1 (*The [Computational Component] does not apply [Threat Data] to the [Communication Component] when [Threat Present] is in the state [Yes]*) could be caused by a faulty algorithm that fails to perceive an existing threat. This scenario is highly relevant to our presented solution, where a neural network is used for threat perception since current object detection neural networks cannot provide the desired accuracy. After creating four different loss scenarios and assigning them to the unsafe control actions that could cause them, another table was created, shown in Fig. 10. In this table, more precise descriptions of the loss scenarios are provided and the individual losses that were defined before the analysis were assigned to the scenarios. Figure 11 visualizes the relations between the different STPA elements. From the action *Threat Data* two unsafe control actions are derived. The unsafe control actions can be traced to loss scenarios under a specific context, which in turn can be traced to causal factors. At the end of the analysis, safety mitigations must be defined to prevent the loss scenarios and meet the associated requirements. For the example of the neural network, verification methods could be applied here. Using automation to collect information from the model and to generate the tables helps to remove error-prone tasks such as the creation of hazard tables. We think using the Model-Based STPA for the development of AI-based systems is useful since automatic generation and updating helps with frequently changing models.

#	Name	Causal Factor	Causal Factor Classification	Causal Factor Source	Loss	Loss Scenario Description	Mitigations
1	Faulty algorithm cannot identify threats	Model of control algorithm	Algorithm	Computational Component	Near Mid Air Collision	A faulty algorithm fails to identify a threat. The result is a near mid air collision.	3 STPA Requirement 0
2	Faulty algorithm mistakenly identifies threat	Model of control algorithm	Algorithm	Computational Component	Dangerous Maneuver Pilot Workload Increase	A faulty algorithm falsely identifies a threat. As a result the pilot executes a dangerous avoidance maneuver.	4 STPA Requirement 1
3	Inadequate Sensordata leads to missing identification	Sensor Data	Input	Sensor	Near Mid Air Collision	Inadequate sensor data prevents correct identifications. The result is a near mid air collision.	4 STPA Requirement 1
4	Wrong Vehicle Information	Vehicle Status	Input	Vehicle Avionics	Dangerous Maneuver Pilot Workload Increase	Wrong vehicle information causes the system to falsely calculate a collision. As a result the pilot executes a dangerous avoidance maneuver.	3 STPA Requirement 0

Fig. 10 Exemplary loss scenarios identified by the Model-Based STPA

C. Scenario-Based Safety Assessment

The behavior of ML-based applications emerges from the data set used for training. Traditional verification methods are not applicable when dealing with ML-based systems. As stated in [39], current certification guidance such as DO-178C (Software Considerations in Airborne Systems and Equipment Certification) [5] and DO-331 (Model-Based

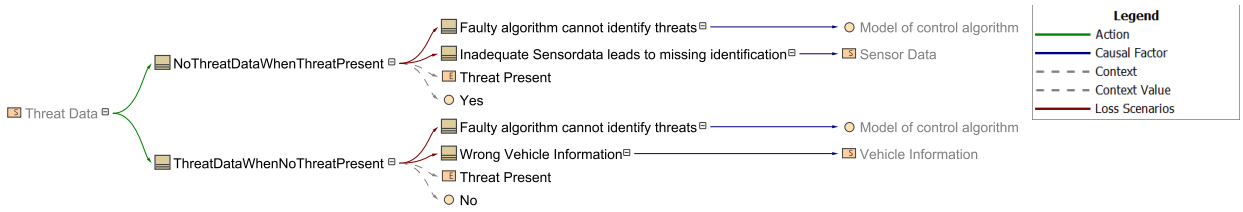

Fig. 11 Relations between STPA elements visualized

Development and Verification Supplement to DO-178C and DO-278A) [40] are facing barriers when it comes to AI-based systems. Further [39] concludes that certification most likely will rely on a combination of different techniques. One applicable technique to evaluate the fulfillment of functional requirements and the absence of unintended behavior is scenario-based testing. Scenario-based testing is an approach to software testing that is trying to assure safety by executing simulations while trying to cover the operational space of the system as well as possible. The automotive domain is already putting great effort into the research of scenario-based testing for automated vehicles [41].

Scenario-based testing came up as an alternative approach to standard distance-based testing for Automated Vehicles [42]. It requires well-designed scenarios that can sufficiently test the capabilities of the automated driving systems [43]. Automated systems using AI are complex and may require infinite use cases to test the complete system. The scenario-based approach should reduce the unlimited test scope to a reasonable number of scenarios [44]. The aim is to compress the test drive into scenarios and thus remove negligible sections without an action or event (e.g. monotonous straight driving) from the validation procedure. This approach adds value to AI-based aviation systems, which due to their ML components, can lead to many possibilities.

A scenario describes the system with the major entities, their initial conditions, a timeline of significant events, and the environment [45]. Scenario development is a process to transform the stakeholder's description of a scenario in plain language to executable specifications that can be used on a simulator. This transformation takes place in three successive stages per the classification given in [46] and [47]. The first one is operational scenarios describing the system's intended use in plain English. The second one is conceptual scenarios which formally model the scenarios by having various elements and their relationships. The third one is executable scenarios described in a machine-readable format and can execute in a simulator.

The Operational Design Domain specifies the range of environmental conditions within which the Automated System can operate. SAE J3016 [48] defines ODD as "The specific conditions under which a given driving automation system or feature thereof is designed to function, including, but not limited to, driving modes". To establish the proper capabilities and limitations of Automated Driving Systems (ADSs), their Operational Design Domain must first be defined. It sets up the benchmark to validate the system. It can be helpful for scenario generation, checking the completeness of scenarios, and even monitoring the system to stay in the required ODD. With AI systems' unpredictable nature, ODD becomes vital for the safety process.

Scenarios in aviation started with the Aviation Scenario Description language, which uses different modelling approaches such as UML / Ecore and System Entity Structures (SES) [49]. It evolved later with applications such as modelling scenario elements of a research flight simulator [50] and drogue detection for an aircraft [51]. In the latest work [31], we used the approach to generate scenarios and the ODD. Figure 12 shows the adapted version of our previous work with new terminology. The testing methodology will be renamed Operation Domain Driven Testing, with the Operational domain metamodel at the meta-level and the operational domain model at the domain level.

This paper further shows how a SysML model is utilized to generate the necessary files to run scenarios for our demonstrator based on the operational domain-driven testing approach. It allows us to easily create a new set of test configuration files, including the system's ODD. The SysML profiling extensions were used for the formal description of the domain model. It uses stereotypes, tagged values, and constraints for metamodeling purposes. The process and the models shown in the rest of the section are still a work in progress, and the illustration depicts our initial findings while working with this approach.

We are using a simplified representation of reality where the following aspects define a scenario:

- **Environmental factors** such as weather, luminosity and time of the day
- **Entities** such as the system description under test, including the camera settings, number of intruders and the aircraft model of each intruder
- **Events** such as the flight path of the system under test as well the direction of intrusion

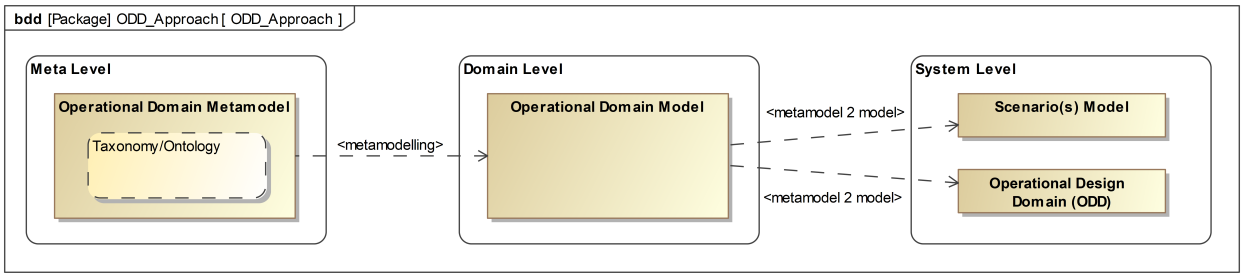

Fig. 12 Operation domain driven testing approach

The domain model shown in Fig. 13 showcases a snippet of the above mentioned aspects. The dotted arrows towards the dummy stereotypes show that other model elements can be added. They have been omitted for a simplified representation. The domain model captures all the possible elements of the simulation for successful testing on the system. The domain model is defined using stereotypes which could lead to further instances representing individual scenarios. As intermediate steps, the block definition diagrams from SysML are used to define the constraints and value ranges for the scenarios and ODD. By selecting the required elements, assigning values to the properties and using the Jython scripts*, we could extract the executables, such as the configuration files for FlightGear, the scenario file, and the machine-readable format for ODD. The following Fig. 14 shows sample files extracted from the domain model. We use the ODD format consistent with the description of ODD in the openODD concept paper [34]. It contains attributes to be included and excluded, along with their types and values. The ODD format in our initial experiments uses XML format to stay consistent with the executables derived from the domain model. The files can be run on the FlightGear simulator, which follows a specific mission sequence outlined in the them. The ODD file could be used for safety assessment of the system executing the scenarios, and its usage is currently beyond the scope of this paper.

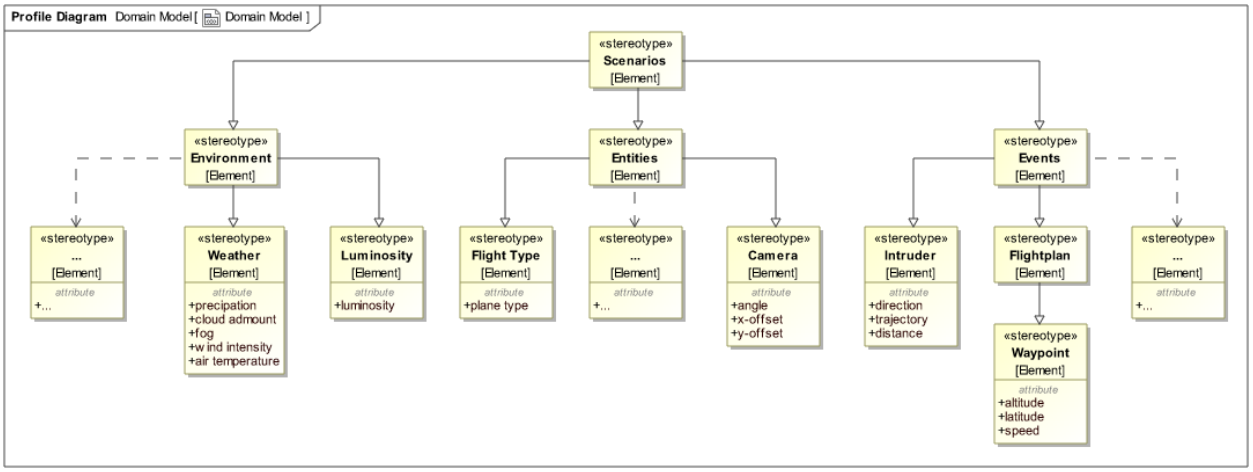

Fig. 13 Sample domain model

V. Conclusion

As evidenced by the creation of new initiatives and the lengthy roadmaps, adoption of AI in aviation systems is a difficult undertaking that requires a high level of expertise in multiple areas. This paper showed our first ideas how MBSE can support the development process of AI-based systems. The presented concepts represent early results of our recently started project MBSE4AI. Using the case study of a threat localization system based on computer vision,

*https://www.jython.org/

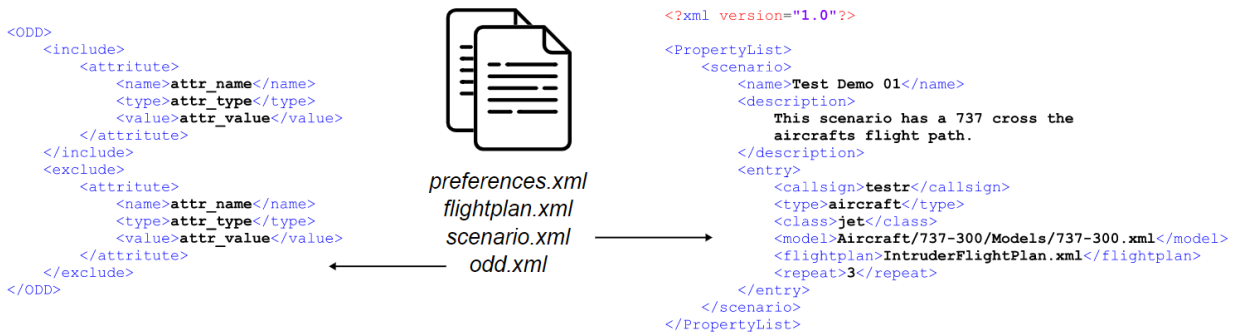

Fig. 14 Output files extracted using the modeling approach

an exemplary systematic modeling process of an AI-based system was shown. With the created models, an MBSA approach called Model-Based STPA was demonstrated to identify hazardous scenarios. Furthermore, scenario-based safety assessment was conducted by applying a systematic approach to generate meaningful test cases for AI-based systems. SysML was used to model the domain of interest and some intermediate models to finally generate scenarios and ODD in XML format. The scenarios were used as configuration inputs to a demonstrator implementation of the threat localization system. The concepts addressed in this paper will be further refined and revisited in future papers. Regarding systematic modeling and the upcoming introduction of the SysML v2, it is interesting to investigate what new capabilities are offered and how these can be used meaningfully in AI development. Regarding the Model-Based STPA, we are planning a combination with the Goal Structuring Notation. Currently, the ODD format is still under research and there may be a few alterations as our research progresses. The scenario generation approach from the domain model is still done manually by selecting the settings and values for each scenario. Automated generation of different scenarios using some criteria will be one of our next steps. We will also conduct research on the topics presented in Fig. 1 that were not included in this paper. In addition, based on future guidelines, we intend to investigate what steps would be required to deploy a threat localization system, such as the one presented in this paper, on a real aircraft.

We currently see a main benefit of using MBSE for development of AI-based systems in the automation possibilities. Since AI development is an iterative process where the systems design can change frequently, automation can support development in different ways. As shown by examples in this paper, it can remove error-prone tasks while assuring complete coverage of the model and generate large amounts of files systematically based on the model, such as test case configurations. Furthermore, when using MBSE small design changes can be easily performed in the model while consistency is maintained. This is another advantage of MBSE for iterative processes. With our submission we want to motivate research to look into the adaption and creation of MBSE concepts to overcome the challenges in the development of AI-based systems. We believe that MBSE offers a multitude of concepts that are beneficial for the development process of safety-critical systems including AI components. Adapting MBSE techniques can help create standardized development methods, which would be an important step toward the goal of safely introducing AI-based systems into aviation.

References

[1] Grace, K., Salvatier, J., Dafoe, A., Zhang, B., and Evans, O., "When will AI exceed human performance? Evidence from AI experts," *Journal of Artificial Intelligence Research*, Vol. 62, 2018, pp. 729–754. https://doi.org/10.1613/jair.1.11222.

[2] Julian, K. D., Kochenderfer, M. J., and Owen, M. P., "Deep Neural Network Compression for Aircraft Collision Avoidance Systems," *Journal of Guidance, Control, and Dynamics*, Vol. 42, No. 3, 2019, pp. 598–608. https://doi.org/10.2514/1.G003724.

[3] Jenn, E., Albore, A., Mamalet, F., Flandin, G., Gabreau, C., Delseny, H., Gauffriau, A., Bonnin, H., Alecu, L., Pirard, J., et al., "Identifying challenges to the certification of machine learning for safety critical systems," *European Congress on Embedded Real Time Systems (ERTS 2020)*, 2020.

[4] S-18 Aircraft and Sys Dev and Safety Assessment Committee, *Guidelines for Development of Civil Aircraft and Systems*, Dec. 2010.

[5] RTCA and EUROCAE, *Software Considerations in Airborne Systems end Equipment Certification*, 2011.

[6] EASA, "EASA Concept Paper: First usable guidance for Level 1 machine learning applications," Tech. rep., Apr. 2021. URL https://www.easa.europa.eu/en/downloads/126648/en.

[7] EuroCAE WG-114/SAE G-34 Artificial Intelligence Working Group, *Artificial Intelligence in Aeronautical Systems: Statement of Concerns*, SAE International, 4 2021. https://doi.org/https://doi.org/10.4271/AIR6988.

[8] EASA, "Artificial Intelligence Roadmap: A human-centric approach to AI in Aviation," Tech. rep., Feb. 2020. URL https://www.easa.europa.eu/en/downloads/109668/en.

[9] Madni, A. M., Boehm, B., Erwin, D., Moghaddam, M., Sievers, M., and Wheaton, M., *Recent Trends and Advances in Model Based Systems Engineering*, Springer International Publishing, 2022. https://doi.org/10.1007/978-3-030-82083-1.

[10] Madni, A. M., and Purohit, S., "Economic analysis of model-based systems engineering," *Systems*, Vol. 7, 2019. https://doi.org/10.3390/systems7010012.

[11] Perry, A. R., "The flightgear flight simulator," *Proceedings of the USENIX Annual Technical Conference*, Vol. 686, 2004.

[12] EASA and Daedalean AG, "Concepts of Design Assurance for Neural Networks (CoDANN)," Tech. rep., Mar. 2020. URL https://www.easa.europa.eu/en/downloads/112151/en.

[13] EASA and Daedalean AG, "Concepts of Design Assurance for Neural Networks (CoDANN) II," Tech. rep., May 2021. URL https://www.easa.europa.eu/en/downloads/128161/en.

[14] Kaakai, F., Dmitriev, K., Adibhatla, S., Baskaya, E., Bezzecchi, E., Bharadwaj, R., Brown, B., Gentile, G., Gingins, C., Grihon, S., et al., "Toward a Machine Learning Development Lifecycle for Product Certification and Approval in Aviation," *SAE International Journal of Aerospace*, Vol. 15, No. 01-15-02-0009, 2022. https://doi.org/10.4271/01-15-02-0009.

[15] DEEL Certification Workgroup, "Machine Learning in Certified Systems," , 2021. https://doi.org/10.48550/arXiv.2103.10529.

[16] Torens, C., Durak, U., and Dauer, J. C., "Guidelines and Regulatory Framework for Machine Learning in Aviation," *AIAA Scitech 2022 Forum*, 2022. https://doi.org/10.2514/6.2022-1132.

[17] Torens, C., Juenger, F., Schirmer, S., Schopferer, S., Maienschein, T. D., and Dauer, J. C., "Machine Learning Verification and Safety for Unmanned Aircraft - A Literature Study," *AIAA Scitech 2022 Forum*, 2022. https://doi.org/10.2514/6.2022-1133.

[18] Estefan, J. A., "Survey of model-based systems engineering (MBSE) methodologies," *Incose MBSE Focus Group*, Vol. 25, No. 8, 2007, pp. 1–12.

[19] incose.org, "Object-Oriented SE Method," , accessed 30.11.2022. URL https://www.incose.org/incose-member-resources/working-groups/transformational/object-oriented-se-method.

[20] Weilkiens, T., *SYSMOD - The Systems Modeling Toolbox: Pragmatic MBSE with SysML*, MBSE4U, 2016.

[21] eclipse.org, "Let yourself be guided with Arcadia," , accessed 30.11.2022. URL https://www.eclipse.org/capella/arcadia.html.

[22] Aleksandraviciene, A., and Morkevicius, A., *MagicGrid® Book of Knowledge, A Practical Guide to System Modeling using MagicGrid from No Magic*, 2nd ed., Vitae Litera: Kaunas, Lithuania, 2021.

[23] Object Management Group, "Risk Analysis and Assessment Modeling Language (RAAML) Libraries and Profiles," , accessed 25.10.2022. URL https://www.omg.org/spec/RAAML/1.0/Beta1/PDF.

[24] Leveson, N. G., *Engineering a safer world: Systems thinking applied to safety*, The MIT Press, 2016.

[25] Leveson, N. G., and Thomas, J., *STPA Handbook*, [Online], 2018. URL https://psas.scripts.mit.edu/home/get_file.php?name=STPA_handbook.pdf.

[26] de Souza, F. G. R., de Melo Bezerra, J., Hirata, C. M., de Saqui-Sannes, P., and Apvrille, L., "Combining stpa with sysml modeling," *2020 IEEE International Systems Conference (SysCon)*, IEEE, 2020, pp. 1–8. https://doi.org/10.1109/SysCon47679.2020.9275867.

[27] Ahlbrecht, A., and Durak, U., "Integrating Safety into MBSE Processes with Formal Methods," *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*, IEEE, 2021, pp. 1–9. https://doi.org/10.1109/DASC52595.2021.9594315.

[28] Ahlbrecht, A., and Bertram, O., "Evaluating System Architecture Safety in Early Phases of Development with MBSE and STPA," *2021 IEEE International Symposium on Systems Engineering (ISSE)*, IEEE, 2021, pp. 1–8. https://doi.org/10.1109/ISSE51541.2021.9582542.

[29] Ahlbrecht, A., and Durak, U., "Model-Based STPA: Towards Safety-Guided Design with Formalization," *2022 IEEE International Symposium on Systems Engineering (ISSE)*, IEEE, 2022, pp. 1–8.

[30] Ahlbrecht, A., and Durak, U., "Model-Based STPA: Enabling Safety Analysis Coverage Assessment with Formalization," *2022 IEEE/AIAA 41st Digital Avionics Systems Conference (DASC)*, IEEE, 2022, pp. 1–10.

[31] Gupta, S., Durak, U., Ellis, O., and Torens, C., "From Operational Scenarios to Synthetic Data: Simulation-Based Data Generation for AI-Based Airborne Systems," *AIAA SCITECH 2022 Forum*, 2022, p. 2103. https://doi.org/10.2514/6.2022-2103.

[32] Jafer, S., Chhaya, B., Durak, U., and Gerlach, T., "Formal scenario definition language for aviation: aircraft landing case study," *AIAA modeling and simulation technologies conference*, 2016, p. 3521. https://doi.org/10.2514/6.2016-3521.

[33] Jafer, S., and Durak, U., "Tackling the complexity of simulation scenario development in aviation," *Proceedings of the Symposium on Modeling and Simulation of Complexity in Intelligent, Adaptive and Autonomous Systems*, 2017, pp. 1–10.

[34] ASAM, "ASAM OpenODD: Concept Paper," https://www.asam.net/standards/detail/openodd/, 2022.

[35] sysml.org, "SysML FAQ: What is SysML? Who created SysML? . . . ," , accessed 18.10.2022. URL https://sysml.org/sysml-faq/.

[36] Wang, C.-Y., Bochkovskiy, A., and Liao, H.-Y. M., "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *arXiv preprint arXiv:2207.02696*, 2022. https://doi.org/10.48550/arXiv.2207.02696.

[37] S-18 Aircraft and Sys Dev and Safety Assessment Committee, *Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment*, Dec. 1996.

[38] Ishimatsu, T., Leveson, N. G., Thomas, J., Katahira, M., Miyamoto, Y., and Nakao, H., "Modeling and hazard analysis using STPA," *Proceedings of the 4th IAASS Conference, Making Safety Matter*, 2010.

[39] Cofer, D., "Unintended Behavior in Learning-Enabled Systems: Detecting the Unknown Unknowns," *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*, IEEE, 2021, pp. 1–7. https://doi.org/10.1109/DASC52595.2021.9594406.

[40] RTCA, *Model-Based Development and Verification Supplement to DO-178C and DO-278A*, 2011.

[41] Riedmaier, S., Ponn, T., Ludwig, D., Schick, B., and Diermeyer, F., "Survey on scenario-based safety assessment of automated vehicles," *IEEE access*, Vol. 8, 2020, pp. 87456–87477. https://doi.org/10.1109/ACCESS.2020.2993730.

[42] Menzel, T., Bagschik, G., and Maurer, M., "Scenarios for development, test and validation of automated vehicles," *2018 IEEE Intelligent Vehicles Symposium (IV)*, IEEE, 2018, pp. 1821–1827. https://doi.org/10.1109/IVS.2018.8500406.

[43] Ma, J., Che, X., Li, Y., and Lai, E. M.-K., "Traffic Scenarios for Automated Vehicle Testing: A Review of Description Languages and Systems," *Machines*, Vol. 9, No. 12, 2021, p. 342. https://doi.org/10.3390/machines9120342.

[44] Weissensteiner, P., Stettinger, G., Rumetshofer, J., and Watzenig, D., "Virtual Validation of an Automated Lane-Keeping System with an Extended Operational Design Domain," *Electronics*, Vol. 11, No. 1, 2021, p. 72. https://doi.org/10.3390/electronics11010072.

[45] Topçu, O., Durak, U., Oğuztüzün, H., and Yilmaz, L., *Distributed simulation: A model driven engineering approach*, Springer, 2016.

[46] Siegfried, R., Laux, A., Rother, M., Steinkamp, D., Herrmann, G., Lüthi, J., and Hahn, M., "Scenarios in military (distributed) simulation environments," *Spring Simulation Interoperability Workshop (S-SIW)*, 2012.

[47] Siegfried, R., Oguztüzün, H., Durak, U., Hatip, A., Herrmann, G., Gustavson, P., and Hahn, M., "Specification and documentation of conceptual scenarios using Base Object Models (BOMs)," *Spring Simulation Interoperability Workshop*, Simulation Interoperability Standards Organization (SISO) San Diego, CA, 2013.

[48] SAE, "J3016_202104 - Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," https://www.sae.org/standards/content/j3016_202104/, 2021.

[49] Jafer, S., Chhaya, B., Zeigler, B. P., and Durak, U., "SES and ecore for ontology-based scenario modeling in aviation scenario definition language (ASDL)," *International Journal of Aviation, Aeronautics, and Aerospace*, Vol. 5, No. 5, 2018, p. 4.

[50] Durak, U., Pruter, I., Gerlach, T., Jafer, S., Pawletta, T., and Hartmann, S., "Using System Entity Structures to model the elements of a scenario in a research flight simulator," *AIAA Modeling and Simulation Technologies Conference*, 2017, p. 1076. https://doi.org/10.2514/6.2017-1076.

[51] Ellis, O., and Durak, U., "Simulation Based Development and Verification of Drogue Detection Algorithms for Autonomous Air to Air Refuelling," *AIAA Scitech 2020 Forum*, 2020, p. 0670. https://doi.org/10.2514/6.2020-0670.